CSA02 – ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

TASK 2 – Choose the Data Types (4 Marks)

Name: BAVADARANI S Register Number: 1912260562

2.1 Data Type Selection

Data item / type	C type	Typical size*	Range / purpose
Meter reading	unsigned short	2 bytes	0 to 65,535 if 16-bit
Household number	unsigned int	4 bytes	0 to 4,294,967,295 if 32-bit
Consumption units	unsigned int	4 bytes	0 to 4,294,967,295 if 32-bit
Tariff / fixed charge	long	Platform-dependent	Integer paise
Energy charge	long	Platform-dependent	Calculated charge in paise
Accumulated total	long long	Typically 8 bytes	Large bill totals
Household category	enum	Implementation-dependent	Named category code
char	char	1 byte	Character or digit
float	float	Typically 4 bytes	Approximate decimal values
double	double	Typically 8 bytes	Higher-precision approximate decimal values

*Sizes are implementation-dependent. Check your system with sizeof. Use <limits.h> for integer limits such as INT_MAX, LONG_MAX, LLONG_MAX and USHRT_MAX, and <float.h> for floating-point limits.

2.2 Why Money Should Be Stored in Paise

Floating-point types cannot represent every decimal fraction exactly. Repeated calculations can introduce rounding errors. Store ■162.50 as 16250 paise and perform tariff calculations using integers. For accumulated totals, verify that the chosen integer type cannot overflow.

2.3 Complete the Code – Meter Rollover

```
#include <stdio.h>
#include <limits.h>

/* Assumes a 16-bit meter counter and at most one rollover. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (unsigned short)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

Expected output on a system where unsigned short is 16-bit:

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

Explanation: For a rollover from 65400 to 120, consumption is (65536 − 65400) + 120 = 136 + 120 = 256 units. The cast converts the result to the 16-bit unsigned range. This assumes USHRT_MAX is 65535 and no more than one rollover occurred.

Compile and run

```
gcc -std=c11 -Wall -Wextra -o task02 task02.c
./task02
```

2.4 Float Accumulation Example

```
#include <stdio.h>

int main(void)
{
    float total = 0.0f;
    for (int i = 0; i < 4000; i++)
        total += 0.10f;
    printf("%.6f\n", total);
    return 0;
}
```

The printed result may differ slightly from 400.000000 because 0.10 is not exactly representable in binary floating point. Storing 10 paise per entry as an integer produces exactly 40,000 paise (■400.00).

2.5 Alternative Concept: Double Rupees

Integer paise gives exact paise arithmetic within the integer range. Double rupees uses typically 8 bytes but small binary floating-point errors may accumulate. Apply rounding consistently at the defined billing boundary. Integer paise is preferred for predictable monetary arithmetic.